FixED Viva Implementation and Chat Session Report

FixED Viva Feature: Detailed Implementation and Debugging Report

Scope: End-to-end Viva feature work completed in this chat session, including architecture, implementation details, incident fixes, and operational outcomes.

1) Objective and Product Scope

The goal was to build a Viva (oral examination / mock interview) feature where a student selects a book/chapter/topic, grants camera/mic permissions, is identity-checked, answers AI-asked oral questions, and receives a detailed dashboard after session completion.

The implemented scope includes setup, live session controls, proctoring checks with warnings and termination policy, post-session scoring, and historical audit retrieval.

2) Initial State Before Implementation

services/viva/main.py existed only as a health-check stub.

No STT/TTS orchestration in Viva service.

No proctoring or identity pipelines.

No Viva data persistence entities in shared DB models.

No Viva route/proxy integration in frontend API and runtime nginx proxy.

3) Backend Implementation Details

3.1 Core service architecture in services/viva/main.py

Startup initializes shared DB schema via init_db().

OpenAI client bootstrap is lazy and keyed by OPENAI_API_KEY.

Cost tracking implemented via _CostTracker and shared cost events.

Session lifecycle handles active/completed/terminated states and timeout guards.

3.2 Implemented endpoints

POST /viva/sessions/start: create session + first question generation.

POST /viva/sessions/{id}/reference-photo: enroll reference face photo.

POST /viva/sessions/{id}/proctor/frame: frame-level presence/match checks with warning escalation.

POST /viva/sessions/{id}/answer: answer ingestion, STT fallback, evaluation, next question generation.

GET /viva/sessions/{id}: current live session state + active question.

POST /viva/sessions/{id}/finish: manual finish and final result generation.

GET /viva/sessions/{id}/results: final dashboard payload.

GET /viva/history/sessions: list past sessions with summary metrics.

GET /viva/sessions/{id}/audit: full audit payload for evaluators.

3.3 Proctoring policy implementation

Presence/identity evaluated on periodic frame submissions.

On failed checks, warning count increments.

Warnings at strike 1-3, auto-termination at strike 4.

Session timeout and per-question timeout termination paths are enforced.

3.4 Prompt and provider layers

services/viva/prompting.py: initial question, follow-up, answer evaluation, summary prompts.

services/viva/providers/voice.py: STT and TTS adapter methods.

services/viva/providers/face.py: face match strategy using base64 fingerprint comparison.

3.5 Data persistence (shared DB)

Added entities in services/shared/db/models.py:

VivaSession
VivaQuestion
VivaTurn
VivaProctorEvent
VivaResult
Also added SQL migration artifact: services/shared/db/migrations/001_viva_tables.sql.
4) Frontend Implementation Details
4.1 New Viva experience
Added frontend/src/pages/VivaPage.jsx with:
Pre-session setup (book/chapter/topic + permissions + reference photo).
Live session question flow (speak question, answer input/STT trigger, submit/finish).
Timers and warning state display.
Results dashboard with score breakdown and AI feedback.
Proctor event timeline and saved frame snapshots.
4.2 API client expansion
frontend/src/services/api.js was extended with Viva methods for session lifecycle, proctoring, results, history, and audit retrieval.
4.3 Routing and navigation
Added route in frontend/src/App.jsx for /viva.
Updated sidebar nav in frontend/src/layouts/LmsLayout.jsx to point Viva item to /viva.
4.4 Runtime proxying
Dev proxy added in frontend/vite.config.js for /api/viva.
Container runtime nginx proxy added in frontend/nginx.conf for /api/viva/ to Viva service.
5) Major Issues Encountered and Fixes Applied
Issue A: Services failed startup with duplicate index error
Error: duplicate ix_viva_results_created_at during schema create.
Cause: index was defined twice for Viva results created_at (implicit + explicit).
Fix: removed duplicate explicit index declaration from model.
Issue B: Viva start endpoint returning 405
Error: POST /api/viva/viva/sessions/start returned 405 behind frontend container.
Cause: runtime nginx config lacked /api/viva/ proxy block.
Fix: added nginx proxy section and rebuilt frontend.
Issue C: OpenAI client init crash (proxies argument)
Error: Client.__init__() got an unexpected keyword argument 'proxies'.
Cause: dependency mismatch between openai and resolved httpx.
Fix: pinned httpx==0.27.2 in Viva requirements and rebuilt Viva container.
Issue D: Submit/finish reliability and warning behavior
Observed: session could appear stuck when backend returned terminated path.
Fixes:
Frontend now handles terminated and no-next-question result transitions robustly.
Proctor checks run immediately + periodic checks (more responsive warning updates).
Face matching strengthened by fingerprinting across distributed base64 windows.

Issue E: Past session history endpoint failed with UUID parsing

Error: route interpreted history as {session_id} UUID.

Cause: route conflict between static and dynamic paths.

Fix: moved history route to /viva/history/sessions and updated frontend client.

6) Persistence and Audit Improvements Added Later

Session start/reference/manual finish events are now also logged as proctor events.

All proctor frames are persisted in event details for evaluator review.

Added session history list for reviewing all Viva/interview attempts.

Added full audit endpoint for evaluator deep review:

session metadata

reference photo

questions asked

turns/answers/evaluations

proctor timeline with frame evidence

final result summary

7) Build and Deployment Changes

services/viva/Dockerfile aligned with multi-service layout (shared code copied + PYTHONPATH set).

docker-compose.yml updated for Viva build path and frontend dependency flow.

Multiple rebuild/restart cycles executed to validate fixes against live container logs.

8) Testing and Validation Performed

Frontend route tests passed via Vitest.

Viva backend unit tests passed via unittest.

Live proxy endpoint checks were run through frontend gateway path.

Container rebuild/restart verifications confirmed service startup after each major fix.

9) Current Functional Outcome

Viva sessions can be started and progressed.

Proctoring warnings and 4th-strike termination are implemented.

Finish/submit transitions are handled reliably in UI and backend.

Results include per-question and AI summary data.

Past sessions and detailed audit artifacts are retrievable.

10) Architectural Notes and Future Hardening

Current frame evidence is stored directly as base64 in DB event JSON; for scale, move to object storage and persist URLs.

Face verification currently uses an internal heuristic; production-grade model/service integration is recommended.

Add role-based evaluator views and access controls for audit payloads.

Add pagination/filtering enhancements and richer search across history.

11) File Areas Touched (High Level)

Backend: services/viva/*

Shared DB: services/shared/db/models.py, services/shared/db/__init__.py, migration SQL

Frontend: frontend/src/pages/VivaPage.jsx, frontend/src/services/api.js, routing/layout files

Infra/runtime: docker-compose.yml, frontend/nginx.conf, frontend/vite.config.js

End of report.